Effectful Computations with Future Conditions: A Monadic Approach to Temporal Specification

AUTHOR NAME, Institution, Country

Pre- and post-conditions are the standard vocabulary for modular program specification, yet they are silent about obligations that span multiple calls. In real Haskell code such obligations are ubiquitous: an acquired mutex must eventually be released; a `beginTx` must eventually reach `commit` or `rollback`; a submitted job must complete within a minimum cost budget. No single pre/post pair can express these properties on its own, because the discharge point lies in an unknown future context.

We present *Pledge*, a Haskell library that realises this idea as a monad transformer: **`Pledge`** `m eff a` wraps an underlying monad `m` and carries, alongside the return value `a`, three annotation fields (`pre`, `post`, `fut`) :: *eff* describing what must have held before, what this action produces, and what must still hold afterwards. Monadic bind propagates these annotations using two algebraic operations: *left-quotient* (\) discharges the first computation's future against the continuation's postcondition, and *right-quotient* (/) combines the continuation's precondition with the first computation's postcondition; the residual obligations are the unmet portions.

The design is parameterised over a **`Composable`** algebra with six operations: sequential composition (·), simultaneous constraint (⊓), their identities (**`empty`**, **`universe`**), left-quotient, and right-quotient. We present four instances: trace protocols via extended regular expressions (RE, with Brzozowski-derivative quotients and a direct LTL_f embedding); Presburger-guarded traces (GRE, checked by Z3); semiring-weighted obligations (WRE, covering probabilistic and min-cost scenarios); and separation-logic heap predicates (SL, with magic wand as quotient). All four share a single bind formula; the monad laws are verified mechanically in Lean 4 from twelve algebraic axioms. We prove a soundness theorem for the RE instance connecting a fully-discharged residual to satisfaction of the obligation by every terminating trace.

CCS Concepts: • **Software and its engineering → Functional languages**; *Data types and structures*; *Software verification and validation*.

Additional Key Words and Phrases: temporal specification, monadic effects, regular expressions, Brzozowski derivatives, left-quotient, right-quotient, Presburger arithmetic, semiring weights, separation logic, resource lifecycle, Haskell, Lean 4

ACM Reference Format:
Author Name. 2026. Effectful Computations with Future Conditions: A Monadic Approach to Temporal Specification. *Proc. ACM Program. Lang.* 1, HASKELL (August 2026), 20 pages.

1 Introduction

Pre- and post-conditions are the workhorse of modular program specification. A pre-condition constrains the state in which an operation may be invoked; a post-condition describes what holds immediately after it returns. Yet many correctness properties span a *sequence* of operations, with arbitrary computation in between.

Author's Contact Information: Author Name, Institution, City, Country, author@institution.edu.

ACM 2475-1421/2026/8-ART
https://doi.org/

Consider three scenarios from the Haskell ecosystem: *(i) Mutex discipline.* An `acquire(m)` must eventually be followed by `release(m)` on every code path. A post-condition records the acquired state but cannot require a future `release`. *(ii) Database savepoints.* A `beginTx` must eventually be committed or rolled back; a pre/post pair on the call cannot carry this obligation forward through intermediate code. *(iii) File handles.* Opening a file in read-mode must be followed by a close, and no write operation may be issued until a write-mode open has been posted.

Existing remedies and their limits. The `bracket` function lexically scopes cleanup but cannot propagate obligations beyond the lexical scope. `ResourceT` [Snoyman 2011] threads cleanup through every bind but models only the cleanup half of an obligation, discharging it uniformly at scope exit. Linear Haskell [Bernardy et al. 2018] enforces "consume exactly once" but cannot express branching discharge or quantitative obligations. Parameterised and indexed monads [Atkey 2009] encode protocol states at the type level but require non-standard syntax and are confined to a single domain.

Relation to prior work. The idea of a *future condition* as a first-class specification component is due to Song et al. [2025], who formalise it at the level of a logical system. This paper's contribution is a concrete, runnable library realisation: a **`Pledge`** monad transformer, an algebraic interface (**`Composable`**) that covers four distinct specification domains, and a Lean 4 mechanisation of the monad laws.

Specification, not enforcement. **`Pledge`** is a *specification* library. A **`Pledge`** `m eff a` value is a computation in the underlying monad `m` that also produces three annotation fields $(pre, post, fut) :: eff$; composing **`Pledge`** values propagates these annotations algebraically. A programmer or toolchain inspects the residual annotations *before* committing to any real effect; **`Pledge`** itself does not instrument or guard a running computation.

Our approach. A value of type **`Pledge`** `m eff a` carries three annotations: a precondition $pre :: eff$ (what must have held before), a postcondition $post :: eff$ (what this action produces), and a future condition $fut :: eff$ (what must hold afterwards). Monadic bind propagates these through two quotient operations on `eff`:

$$pre(p \gg\!= q) = pre(p) \sqcap (pre(q) \,/\, post(p)) \tag{1}$$

$$post(p \gg\!= q) = post(p) \cdot post(q) \tag{2}$$

$$fut(p \gg\!= q) = (fut(p) \setminus post(q)) \sqcap fut(q) \tag{3}$$

Here $\setminus$ is the *left-quotient* (strips an observed prefix from a pending obligation) and $/$ is the *right-quotient* (checks whether a produced postcondition satisfies the continuation's precondition). A fully-discharged residual — $pre = \neg\varnothing$ and $fut = \neg\varnothing$ — is the certificate that the composed program is contract-correct.

The **`Composable`** typeclass abstracts six operations: $(\cdot, \sqcap, \varepsilon, \top, \setminus, /)$. It admits four independent instances:

- *(Trace protocol,* RE*)* Extended regular expressions over a typed event alphabet, with complement as a first-class constructor. Quotients are implemented via Brzozowski derivatives; LTL_f operators embed directly without automaton construction.
- *(Arithmetic bounds,* GRE*)* Each RE is paired with a Presburger arithmetic predicate over heap addresses, checked by Z3 via SBV. One annotation simultaneously enforces a protocol ordering and an arithmetic bound.

• *(Quantitative obligation,* WRE*)* Boolean membership is generalised to a semiring weight. The probability semiring captures "complete with probability $\geq p$"; the tropical semiring gives minimum-cost scheduling obligations.
• *(Heap ownership,* SL*)* Separation-logic heap predicates with separating conjunction as sequential composition and magic wand as quotient. Future conditions describe what heap ownership must hold on completion.

Contributions.

(1) We define the `Pledge` monad transformer (§3), parameterised over a `Composable` algebra, and verify that the monad laws hold for the (*ret*, *post*, *fut*) triple via a Lean 4 mechanisation from twelve algebraic axioms (§10).
(2) We instantiate the algebra to extended regular expressions with Brzozowski-derivative quotients (§5), embed LTL_f directly (§2), and prove a soundness theorem connecting a fully-discharged residual to trace membership (§5.1).
(3) We extend the RE instance to Presburger-guarded traces (§6), semiring-weighted traces (§7), and separation-logic heap predicates (§8).
(4) We demonstrate the approach on five case studies across all four instances (§9), showing that protocol violations, arithmetic overflows, quantitative shortfalls, and ownership errors all surface as non-trivial residual annotations.

2 Background

2.1 Extended Regular Expressions over Typed Events

Events are the atomic units of observation. An `Event t` carries a name and a typed payload:

```
data Term    = Str String | Num Int | List [Term]
data Event t = Atom String t   -- concrete event, e.g. Atom "acquire" (Num 1)
             | Wildcard        -- matches any single event
```

The type parameter `t` allows the payload to be any type, including runtime values such as file handles or thread identifiers. The specification language is an extended regular expression type:

```
data RE t = Bot | Epsilon | Single (Event t)
          | Seq (RE t) (RE t) | Or (RE t) (RE t)
          | And (RE t) (RE t) | Star (RE t) | Not (RE t)
```

The constructors are standard: $\varnothing$ (`Bot`), ε (`Epsilon`), single-event match (`Single`), concatenation (`Seq`), union (`Or`), intersection (`And`), Kleene star (`Star`), and complement (`Not`). `Wildcard` matches any single event. We include `And` as a primitive so its derivative is computed directly without three extra `Not` nodes.

Five derived forms appear throughout the paper:

$$\begin{aligned}
\texttt{anything} &\triangleq \neg\varnothing && \text{(universal language)}\\
\texttt{finally}(e) &\triangleq \texttt{anything} \cdot e \cdot \texttt{anything} && (e \text{ must eventually occur})\\
\texttt{globally}(e) &\triangleq e^* && (e \text{ at every step})\\
\texttt{never}(e) &\triangleq \neg\texttt{finally}(e) && (e \text{ must never occur})\\
\texttt{noUntil}(e, g) &\triangleq \neg\big((\Sigma \setminus \{g\})^* \cdot e \cdot \texttt{anything}\big) && (e \text{ must not occur before } g)
\end{aligned}$$

`previously`(e) is an alias for `finally`(e) used in `pre` slots to assert that e occurred somewhere in the preceding trace; the underlying RE is identical, but the name signals intent.

2.2 Algebraic Complement and Brzozowski Derivatives

The Brzozowski derivative [Brzozowski 1964] $\partial_a(r)$ computes the language of continuations after reading event a. The key law for complement is:

$$\partial_a(\neg r) \;=\; \neg\big(\partial_a(r)\big) \tag{4}$$

Together with the De Morgan laws and double-negation elimination, this lets complement be propagated purely algebraically, without building a DFA. The nullability function $\nu(r)$ (= `True` iff $\varepsilon \in L(r)$) satisfies $\nu(\neg r) = \neg\,\nu(r)$.

```
nullable :: RE t → Bool
nullable (Not r)     = not (nullable r)
nullable (Star _)    = True
nullable (Seq r1 r2) = nullable r1 && nullable r2
nullable (Or  r1 r2) = nullable r1 || nullable r2
nullable (And r1 r2) = nullable r1 && nullable r2
nullable Epsilon     = True
nullable _           = False

derivative :: Eq t ⇒ Event t → RE t → RE t
derivative e (Not r)      = Not (derivative e r)   -- da(Not r) = Not(da(r))
derivative e (Single p)   = if subsumesEvent e p then Epsilon else Bot
derivative e (Seq r1 r2)
    | nullable r1 = Or (Seq (derivative e r1) r2) (derivative e r2)
    | otherwise   = Seq (derivative e r1) r2
derivative e (Or  r1 r2)  = Or  (derivative e r1) (derivative e r2)
derivative e (And r1 r2)  = And (derivative e r1) (derivative e r2)
derivative e (Star r)     = Seq (derivative e r) (Star r)
derivative _ _            = Bot
```

The `firstWith` function returns the events from a supplied finite alphabet Σ_0 that can begin a word in $L(r)$. Complement requires care: naively returning $\mathit{first}(\neg r) = \mathit{first}(r)$ is unsound. The correct characterisation is:

$$e \in \mathit{first}(\neg r, \Sigma_0) \iff e \in \Sigma_0 \;\wedge\; \lceil \partial_e(r) \rceil \neq \neg\varnothing \tag{5}$$

That is, e can open a word in $L(\neg r)$ iff, after consuming e, the residual $\partial_e(r)$ does not cover the entire language.

2.3 Embedding LTL$_f$

Because complement is a first-class constructor, the standard LTL$_f$ operators translate directly into RE without automaton construction. Table 1 gives the full mapping.

3 The Pledge Monad

3.1 The `Composable` Class

We abstract the specification algebra via a type class with six operations:

```
class Composable a where
  concatenation :: a → a → a   -- (·), identity: empty
  conjunction   :: a → a → a   -- (⊓), identity: universe
  empty         :: a           -- identity for (·)
  universe      :: a           -- identity for (⊓)
```

Table 1. LTL_f to RE translation. $\Sigma^1 \triangleq$ `Single Wildcard`, $\neg\varnothing \triangleq \neg\varnothing$.

LTL formula	RE $[\![\cdot]\!]$	
$\top$	$\neg\varnothing$	
$\bot$	$\varnothing$	
e	$\{e\}$	
$\neg\phi$	$\neg[\![\phi]\!]$	
$\phi \wedge \psi$	$[\![\phi]\!] \cap [\![\psi]\!]$	
$\phi \vee \psi$	$[\![\phi]\!] \cup [\![\psi]\!]$	
$X\,\phi$	$\Sigma^1 \cdot [\![\phi]\!]$	
$F\,\phi$	$\neg\varnothing \cdot [\![\phi]\!]$	
$G\,\phi$	$\neg(\neg\varnothing \cdot \neg[\![\phi]\!])$	
$\phi\,U\,\psi$	$\mathit{step}(\phi)^* \cdot [\![\psi]\!]$	(ϕ propositional)

```
leftQuotient  :: a → a → a   -- dividend \ divisor
rightQuotient :: a → a → a   -- dividend / divisor
```

The library provides Unicode infix aliases: `(·) = concatenation`, `(⊓) = conjunction`, `a \ b = leftQuotient b a`, and `p / q = rightQuotient q p`. The swapped arguments in the infix definitions are intentional: *fut* \ *post* means "left-quotient of *fut* by *post*," i.e., strip *post* from the left of *fut*. Similarly, *pre* / *post* means "right-quotient of *pre* by *post*," i.e., strip *post* from the right of *pre*.

Algebraic roles. `concatenation` sequences effects; `conjunction` combines simultaneous constraints; `empty` and `universe` appear in `pure`. The two quotient operations implement *obligation discharge*. Intuitively:

- *fut* \ *post*: given that the continuation has posted *post*, what of the original future obligation *fut* remains unmet from the left?
- *pre* / *post*: given that the current action has produced *post*, what portion of the next action's precondition *pre* is not yet satisfied from the right?

Two base cases anchor both quotients: $r \setminus \varepsilon = r$ (nothing observed, obligation unchanged) and $r \setminus \top = \top$ (universal postcondition covers anything). The symmetric laws hold for /.

Component-wise lifting. The instance for pairs lifts `Composable` component-wise, enabling multiple independent specification dimensions to be combined:

```
instance (Composable a, Composable b) ⇒ Composable (a, b) where
  concatenation (a1,b1) (a2,b2) = (concatenation a1 a2, concatenation b1 b2)
  leftQuotient  (a1,b1) (a2,b2) = (leftQuotient  a1 a2, leftQuotient  b1 b2)
  -- etc.
```

3.2 The Pledge Monad Transformer

```
newtype Pledge m eff a = Pledge { runPledge :: m (a, eff, eff, eff) }
```

A value of type `Pledge m eff a` is a computation in the underlying monad `m` that produces a 4-tuple $(a, \mathit{pre}, \mathit{post}, \mathit{fut})$: the return value and three annotation fields. When `m = IO`, the annotations can be built from actual runtime values (e.g., a freshly allocated file handle), because the 4-tuple is produced by the `IO` action itself.

Two combinators support interoperability:

```
-- lift a plain m-action with trivial annotations
liftPledge :: (Composable eff, Applicative m) ⇒ m a → Pledge m eff a
liftPledge ma = Pledge $ fmap (, universe, empty, universe) ma

-- run once and collect all four components
inspect :: Functor m ⇒ Pledge m eff a → m (PledgeResult eff a)
inspect (Pledge ma) = fmap (\(a,pre,post,fut) → PledgeResult a pre post fut) ma
```

`liftPledge` embeds any `m`-action as a `Pledge` with no precondition, no postcondition, and no future obligation. `inspect` runs the action exactly once and packages the result; callers should prefer `inspect` over the individual field accessors (`getPre`, `getPost`, `getFut`) when `m` has side effects.

3.3 Monad Instance

`pure` introduces a value with no effect and no obligation:

```
pure x = Pledge $ pure (x, universe, empty, universe)
```

Bind sequences two computations and propagates annotations according to Equations (1)–(3):

```
Pledge ma ≫= g = Pledge $ do
    (a, preA, postA, futA) <- ma
    (b, preB, postB, futB) <- runPledge (g a)
    return (b,  preA ⊓ (preB / postA),
                postA · postB,
               (futA \ postB) ⊓ futB)
```

Future condition. The expression $fut(A) \setminus post(B)$ computes the residual of A's future obligation after observing B's postcondition: events discharged by $post(B)$ are removed, and the remainder is conjoined with B's own future obligation. When the result normalises to $\top$, all future obligations are discharged.

Precondition. The expression $pre(B) \,/\, post(A)$ is the *residual precondition*: the part of B's precondition not yet satisfied by A's postcondition. If $post(A)$ does not cover $pre(B)$, the residual is $\varnothing$, flagging the violation. If it fully covers $pre(B)$, the residual is ε and the combined precondition reduces to $pre(A)$, matching the standard Hoare sequencing rule. As discussed in §10, the `pre` field follows a Hoare-rule discipline rather than strict monad-law equality; the three monad laws are proved for the $(ret, post, fut)$ triple.

3.4 How the Specification Meets Real Effects

Because annotations live inside the underlying `m`, the `Pledge` monad transformer integrates naturally with real effectful code. Two usage patterns are common.

Pure annotations (pure composition). When `m = IO` and the specification primitives use `Pledge \$ return (val, pre, post, fut)`, annotations are pure Haskell values and the `IO` action does nothing. Composing such primitives with ≫= builds a specification term; the programmer inspects `pre` and `fut` of the result to check contract-correctness, then separately executes the real effectful code.

Live-handle annotations. When `m = IO` and the underlying action is real, the annotations can reference actual runtime values. For example:

```
openFile :: FilePath → IO.IOMode → Pledge IO (RE IO.Handle) IO.Handle
openFile fn mode = Pledge $ do
```

```
h <- IO.openFile fn mode            -- actual IO
return (h, universe,
        Single (Atom "open" h),     -- post uses the real handle h
        finally (Atom "close" h))   -- fut requires closing h specifically
```

Here `h :: IO.Handle` is the actual file handle produced at runtime, embedded directly into the `RE IO.Handle` annotations. The left-quotient machinery then propagates this concrete handle through subsequent binds, so a residual `finally(close h)` names the specific unclosed handle. This is the mechanism by which dynamic runtime values appear in residual obligations without requiring a data-dependent `a → eff` type for the future field.

4 Monad Laws

Every monad must satisfy three equational laws. **`Pledge`**'s four annotation fields do not all satisfy the laws under ordinary equality: *pre* follows the Hoare-rule discipline of Equation (1), which deliberately departs from left-identity when a precondition violation is detected (the violation must remain visible after further binding). What we prove, and mechanise in Lean 4, is that the triple $(ret, post, fut)$ satisfies the three laws exactly. The *pre* field is carried alongside as a separate contract-checking annotation, analogous to a writer-monad log that is inspected but not required to satisfy the monad laws on its own.

The laws hold assuming the twelve axioms in Table 5, which we state abstractly over the **`Composable`** algebra.

Table 2. The twelve **`Composable`** axioms required to prove the monad laws for the $(ret, post, fut)$ triple.

Label	Axiom	Used for
S1	$\varepsilon \cdot a = a$	Law 1, post
S2	$a \cdot \varepsilon = a$	Law 2, post
S3	$(a \cdot b) \cdot c = a \cdot (b \cdot c)$	Law 3, post
C1	$a \sqcap b = b \sqcap a$	Law 2, pre / fut
C2	$(a \sqcap b) \sqcap c = a \sqcap (b \sqcap c)$	Law 3, pre / fut
C3	$\top \sqcap a = a$	Laws 1, 2
L1	$r \setminus \varepsilon = r$	Law 2, fut
L2	$\top \setminus r = \top$	Law 1, fut
L3	$r \setminus (a \cdot b) = (r \setminus a) \setminus b$	Law 3, fut
L4	$(a \sqcap b) \setminus c = (a \setminus c) \sqcap (b \setminus c)$	Law 3, fut
R1	$a \,/\, \varepsilon = a$	Law 1, pre
R2	$\top \,/\, r = \top$	Law 2, pre
R3	$(a \,/\, b) \,/\, c = a \,/\, (c \cdot b)$	Law 3, pre
R4	$(a \sqcap b) \,/\, c = (a \,/\, c) \sqcap (b \,/\, c)$	Law 3, pre

We verify all three laws using these axioms. In each case we show that both sides of the law produce equal $(ret, post, fut)$ triples.

Left identity: `pure x >>= f = f x`. Let $p =$ `pure` $x = (x, \top, \varepsilon, \top)$ and $q = f\,x$. By Equations (1)–(3):

$$ret : ret(q).\checkmark$$
$$post : \varepsilon \cdot post(q) = post(q). \quad \text{(S1)}\checkmark$$
$$fut : (\top \setminus post(q)) \sqcap fut(q) = \top \sqcap fut(q) = fut(q). \quad \text{(L2, C3)}\checkmark$$

For *pre*: $pre(q) \,/\, \varepsilon = pre(q)$ (R1), so $pre = \top \sqcap pre(q) = pre(q)$ (C3). ✓

Right identity: `e >>= pure = e`. Let $q = \mathsf{pure}(ret(e)) = (ret(e), \top, \varepsilon, \top)$.

$$ret : ret(e).\checkmark$$
$$post : post(e) \cdot \varepsilon = post(e). \quad \text{(S2)}\checkmark$$
$$fut : (fut(e) \setminus \varepsilon) \sqcap \top = fut(e) \sqcap \top = \top \sqcap fut(e) = fut(e). \quad \text{(L1, C1, C3)}\checkmark$$

Associativity. Let $r_1 = ret(e)$, $r_2 = ret(f\, r_1)$. Both sides yield $ret(g\, r_2)$ and $post(e) \cdot post(f\, r_1) \cdot post(g\, r_2)$ (by S3). For the future, unfolding Equation (3) on the LHS and applying L4 then L3 gives:

$$\big((fut(e) \setminus post(f\, r_1)) \setminus post(g\, r_2)\big) \sqcap (fut(f\, r_1) \setminus post(g\, r_2)) \sqcap fut(g\, r_2)$$

which equals the RHS by C2 and L3. ✓ For *pre*, applying R4 then R3 gives the same result on both sides by C2. ✓

Lean 4 mechanisation. The three law proofs are mechanised in Lean 4 in the file `Formalization/PledgeMonad` using an abstract **`Composable`** structure parametrised over the twelve axioms of Table 5. The Lean file carries no `sorry`; all proofs are term-mode derivations from the stated axioms. A companion file `Formalization/REComposableLaws.lean` verifies that the RE instance satisfies all twelve axioms.

5 The RE Instance

The **`Composable`** `(RE t)` instance maps the algebra operations onto RE constructors and implements the quotients via Brzozowski derivatives. The left-quotient $r_2 \setminus r_1$ — "what of r_2 remains after r_1 is consumed from the left" — is computed by Antimirov partial derivatives on r_2, which keep intermediate terms smaller than the Brzozowski derivative while producing the same language.

```
instance Eq t ⇒ Composable (RE t) where
  concatenation = Seq
  conjunction   = And
  empty         = Epsilon
  universe      = Not Bot        -- = Sigma*
  leftQuotient  = reLeftQuotient
  rightQuotient = reRightQuotient
```

The right-quotient is computed via reversal: $r_2 / r_1 = rev(rev(r_2) \setminus rev(r_1))$, where $rev(r)$ reverses the sequences accepted by r. A normaliser applies the laws in Table 3 to simplify RE terms, exploiting the complement constructor to push `Not` inward via De Morgan's laws.

Table 3. Normalisation laws applied by `normalize`. All rules are applied recursively bottom-up.

Law	Constructor
$\varepsilon \cdot r = r$, $r \cdot \varepsilon = r$	`Seq`
$\varnothing \vee r = r$, $r \vee \neg\varnothing = \neg\varnothing$	`Or`
$\varnothing \wedge r = \varnothing$, $r \wedge \neg\varnothing = r$	`And`
$\neg\neg r = r$	`Not`
$\neg(r_1 \vee r_2) = \neg r_1 \wedge \neg r_2$	`Not`
$\neg(r_1 \wedge r_2) = \neg r_1 \vee \neg r_2$	`Not`
$\neg\varnothing = \neg\varnothing$, $\neg\neg\varnothing = \varnothing$	`Not`
$\varnothing^* = \varepsilon$, $\varepsilon^* = \varepsilon$	**`Star`**

Termination. The left-quotient recursion terminates because the set of distinct normalised derivatives of any regular expression is finite — a classical result of Brzozowski [Brzozowski 1964], extended to complement by Owens et al. [Owens et al. 2009]. Each recursive call is made with the normalised derivative of the previous operands, so the recursion explores a path in this finite set and must eventually reach a base case.

5.1 Soundness

A fully-discharged residual ($\neg\varnothing$ in *fut*, $\neg\varnothing$ in *pre*) serves as a certificate of contract-correctness. We now prove that this certificate is sound for the RE instance.

For an RE r and a finite trace w, write $w \models r$ for $w \in L(r)$ and $\partial_w(r)$ for the iterated derivative.

Lemma 5.1 (Derivative correctness). *For all r and w: $w \models r$ iff $\nu(\partial_w(r)) = \texttt{True}$.*

Proof. By induction on w. The base case is the definition of ν. For $w = a \cdot w'$: $w \models r$ iff $w' \models \partial_a(r)$ (standard derivative correctness [Brzozowski 1964], extended to `Not` by Equation (4) and to `And` by the direct derivative clause); the inductive hypothesis on w' and $\partial_a(r)$ gives $\nu(\partial_{w'}(\partial_a(r))) = \nu(\partial_w(r))$. □

Theorem 5.2 (Soundness of RE-residual checking). *Let e :: `Pledge RE` a be a specification term built from `pure` and $\gg\!=$ over `RE`-annotated primitives, and let w be any finite trace whose event sequence matches the `post` fields of the invoked primitives in order. If $\lceil fut(e) \rceil = \neg\varnothing$ and $\lceil pre(e) \rceil = \neg\varnothing$, then w satisfies every future condition and every precondition named by the primitives composing e.*

Proof sketch. By induction on the structure of e. For $e = \texttt{pure}\ x$, both residuals are $\neg\varnothing$ by construction and there are no obligations; the claim holds vacuously. For $e = p \gg\!= f$: by Equation (3), $fut(e) = \neg\varnothing$ iff $(fut(p) \setminus post(f\ r)) \sqcap fut(f\ r) = \neg\varnothing$, i.e. $fut(p) \setminus post(f\ r) = \neg\varnothing$ and $fut(f\ r) = \neg\varnothing$. By Lemma 5.1, $fut(p) \setminus post(f\ r) = \neg\varnothing$ means that every trace matching $post(f\ r)$ discharges the entire obligation $fut(p)$; the inductive hypothesis on p and $f\ r$ then gives that the corresponding trace segments satisfy their respective obligations. The precondition case is symmetric using Equation (1). □

Theorem 5.2 applies to the RE instance because the proof relies on derivative correctness for regular languages. The GRE instance inherits soundness for its RE component and requires the Presburger side to be checked by a sound decision procedure (Z3), which we take as given. The SL instance does not admit an analogous soundness theorem as it stands, because its quotient records the magic wand symbolically without a semantic heap model; connecting residual predicates to a solver is identified in §12 as the natural next step.

6 The GuardedRE Instance

The RE instance reasons about event traces but is silent about arithmetic state. The GRE instance closes this gap: a guarded regular expression pairs a trace RE with a Presburger arithmetic predicate over heap addresses, so one annotation simultaneously enforces a protocol ordering and a numeric bound.

6.1 The GuardedRE Type

```
data GuardedRE a = GuardedRE PPred (RE a)
```

A state (heap, trace) satisfies `GuardedRE p r` iff heap $\models p$ and trace $\in L(r)$. The two sides are independent: p is a static constraint, while r advances through derivatives as events arrive.

Smart constructors lift a pure RE or pure predicate:

```
fromRE    :: RE a    → GuardedRE a
fromPPred :: PPred   → GuardedRE a
```

6.2 The `Composable GuardedRE` Instance

```
instance Eq a ⇒ Composable (GuardedRE a) where
  concatenation (GuardedRE p1 r1) (GuardedRE p2 r2) =
      GuardedRE (normalizePPred (PAnd p1 p2)) (normalize (Seq r1 r2))
  conjunction = conjoin
  empty    = GuardedRE PTrue Epsilon
  universe = GuardedRE PTrue (Not Bot)
  leftQuotient  (GuardedRE p1 r1) (GuardedRE p2 r2) =
      GuardedRE (normalizePPred (PAnd p1 p2)) (normalize (reLeftQuotient r1 r2))
  rightQuotient (GuardedRE p1 r1) (GuardedRE p2 r2) =
      GuardedRE (normalizePPred (PAnd p1 p2)) (normalize (reRightQuotient r1 r2))
```

Both quotient operations conjoin the two heap predicates: the residual must respect constraints from both operands. The RE component uses the same derivative-based quotients as the plain RE instance.

6.3 Presburger Predicates

Predicates are built over a linear arithmetic expression language:

```
data PExpr = Lit Int | ValAt Addr | Add PExpr PExpr | Mul Int PExpr
data PPred = PTrue | PFalse
           | PLt PExpr PExpr | PLe PExpr PExpr | PEq PExpr PExpr
           | PGt PExpr PExpr | PGe PExpr PExpr
           | PNot PPred | PAnd PPred PPred
```

`ValAt a` reads the value at heap address `a`. The normaliser `normalizePPred` flattens `PAnd` trees, removes duplicates, substitutes equalities of the form $h[a] = k$, evaluates literal comparisons, and collapses $\neg$**true** to `PFalse`. A predicate reaching the solver that is syntactically `PFalse` is rejected immediately without an SMT call. Non-trivial residual predicates are discharged by Z3 via SBV.

7 The WeightedRE Instance

The RE and GRE instances are Boolean: a future condition is either satisfied or violated. The WRE instance generalises membership to a weight from a semiring, enabling probabilistic and quantitative obligations.

7.1 The Semiring Class

```
class (Eq w, Show w) ⇒ Semiring w where
  szero :: w;  sone :: w
  sadd  :: w → w → w    -- choice / union
  smul  :: w → w → w    -- sequence / composition
```

Two concrete instances capture quantitative scenarios:

Instance	`szero`	`sone`	`sadd`	`smul`
`Prob` (probability)	0	1	$\min(p+q,\,1)$	$p \times q$
`Tropical` (min-cost)	$+\infty$	0	min	$+$

7.2 The WRE Type

```
data WRE w t
    = WBot | WEps w | WSingle w (Event t)
    | WSeq (WRE w t) (WRE w t) | WAdd (WRE w t) (WRE w t)
    | WAnd (WRE w t) (WRE w t) | WStar (WRE w t)
```

The generalised nullable function accumulates the weight assigned to the empty trace:

```
wNullable :: Semiring w ⇒ WRE w t → w
wNullable (WEps w)     = w
wNullable (WSeq r1 r2) = smul (wNullable r1) (wNullable r2)
wNullable (WAdd r1 r2) = sadd (wNullable r1) (wNullable r2)
wNullable (WAnd r1 r2) = smul (wNullable r1) (wNullable r2)
wNullable (WStar _)    = sone
wNullable _            = szero
```

Smart constructors `wFinally w e` and `wGlobally w e` define weighted analogues of `finally`(e) and `globally`(e):

```
wTop     = WStar (WSingle sone Wildcard)
wFinally w e = WSeq wTop (WSingle w e)
wGlobally w e = WStar (WSingle w e)
```

The `Composable (WRE w t)` instance maps `concatenation` to `WSeq`, `conjunction` to `WAnd`, and implements the quotients via weighted Brzozowski derivatives.

Absence of weighted complement. `WRE` does not include a `WNot` constructor. Boolean negation ($0 \leftrightarrow 1$) has no canonical generalisation to an arbitrary semiring, so the LTL_f operators that rely on complement (G, U, `never`) are unavailable. The operators `wFinally` and `wGlobally` are defined directly via `WStar` and `WSeq` without complement, covering the common quantitative patterns.

8 The SL Instance

The `Composable` typeclass is not inherently tied to trace-based specifications. We demonstrate its generality with a fourth instance over *separation-logic heap predicates*, where future conditions describe what heap ownership must hold upon completion.

8.1 Structural Parallel with RE

Table 4 shows the role each SL construct plays in the `Composable` interface.

Table 4. Structural parallel between the four `Composable` instances.

Operation	RE	GRE	WRE_w	SL
`concatenation`	$r_1 \cdot r_2$	$(p_1 \wedge p_2,\ r_1 \cdot r_2)$	`WSeq`	$P * Q$
`conjunction`	$r_1 \wedge r_2$	$(p_1 \wedge p_2,\ r_1 \wedge r_2)$	`WAnd`	$P \wedge Q$
`empty`	ε	$(\mathbf{true}, \varepsilon)$	`WEps sone`	**emp**
`universe`	$\neg\varnothing$	$(\mathbf{true}, \neg\varnothing)$	`wTop`	$\top$
`leftQuotient`	Brzozowski derivative	quotient (RE) + $\wedge$ (PPred)	weighted derivative	$P \mathrel{-\!\!*} Q$
`rightQuotient`	reversal + left-quotient	idem	idem	$P \mathrel{-\!\!*} Q$

8.2 The SL Predicate Type

```
data SL
    = Emp              -- empty heap (identity for SepStar)
    | Top              -- any heap  (identity for Conj)
    | Pure PPred       -- pure arithmetic constraint (heap-independent)
    | Cell Addr Val    -- singleton: address a holds value v
    | SepStar SL SL    -- P * Q  separating conjunction
    | Conj   SL SL     -- P /\ Q ordinary conjunction
    | Wand   SL SL     -- P -* Q magic wand (residual)
```

8.3 The `Composable SL` Instance

```
instance Composable SL where
  concatenation = SepStar
  conjunction   = Conj
  empty         = Emp
  universe      = Top
  -- emp -* Q = Q  (nothing provided, obligation unchanged)
  leftQuotient Emp q = q
  -- Top is trivially universal; any Q is discharged.
  leftQuotient Top _ = Emp
  leftQuotient p   q = Wand p q   -- symbolic magic wand
  -- SepStar is commutative, so left- and right-quotients coincide.
  rightQuotient = leftQuotient
```

The two base cases mirror the RE instance: $\mathbf{emp} \mathbin{-\!\!*} Q = Q$ (providing no heap leaves the obligation unchanged) and $\top \mathbin{-\!\!*} Q = \mathbf{emp}$ (any postcondition discharges the obligation). The general case records the wand *symbolically* for later evaluation.

Symbolic recording, not semantic discharge. The five normalisation rules of `normalizeSL` reduce syntactically trivial wands, but the general `Wand p q` case is retained as a data structure: no heap model is consulted and no solver is invoked. The SL instance therefore records obligations and propagates them correctly through bind; it does not check whether the accumulated residual is satisfiable. Consequently, Theorem 5.2's style of guarantee does not extend to SL as it stands. Connecting residual predicates to an external solver (Z3 via `sbv` or a dedicated SL procedure such as Smallfoot) is identified in §12 as the concrete next step.

9 Examples

We demonstrate `Pledge` across five case studies covering all four instances. Each example defines primitive operations annotated with `pre`, `post`, and `fut`, composes them in the `Pledge` monad, and inspects the residual. A *fut* of $\neg\varnothing$ (or `wNullable = sone` for WRE) and a *pre* of $\neg\varnothing$ together certify temporal correctness (Theorem 5.2 for RE). The examples below focus on programs whose future conditions are genuinely non-trivial, since a $\top$ future throughout would make no use of the propagation machinery.

9.1 Mutex Lifecycle (RE Instance)

An acquired mutex must eventually be released, and a release requires a prior acquire. The event alphabet is `Term`; mutex identifiers are `Num mid`.

```
acquire :: Int → Pledge IO (RE Term) ()
acquire mid = Pledge $ return
    ( ()
    , universe                                      -- pre: none
    , Single (Atom "acquire" (List [Num mid]))      -- post
    , finally (Atom "release" (List [Num mid])) )   -- fut: must release

release :: Int → Pledge IO (RE Term) ()
release mid = Pledge $ return
    ( ()
    , Single (Atom "acquire" (List [Num mid]))  -- pre: must have acquired
    , Single (Atom "release" (List [Num mid]))  -- post
    , universe )                                -- fut: none
```

The four example programs below and their residuals illustrate the key scenarios:

```
-- Good: acquire then release
safeSection = do { acquire 1; release 1 }

-- Good: nested locks, released in reverse order
nestedLocks = do { acquire 1; acquire 2; release 2; release 1 }

-- Bad future: lock 1 never released
lockLeak = do { acquire 1; acquire 2; release 2 }

-- Bad pre: release without prior acquire
releaseWithoutAcquire = release 1
```

Program	Residual *fut*	Residual *pre*
`safeSection`	¬∅	¬∅
`nestedLocks`	¬∅	¬∅
`lockLeak`	`finally(release(1))`	¬∅
`releaseWithoutAcquire`	¬∅	∅

For `lockLeak`, the residual future names lock 1 specifically because the future condition of the first `acquire` is discharged by the `release 2` postcondition only for lock 2; lock 1's obligation `finally(release(1))` passes through the left-quotient unchanged. For `releaseWithoutAcquire`, `pre` collapses to ∅: the `Single(acquire(1))` precondition of `release` is never posted, so the right-quotient `Single(acquire(1))` / $\varepsilon = \varnothing$.

9.2 Database Transactions (RE Instance)

A transaction begun with `beginTx` carries a *disjunctive* future condition requiring eventual commit or rollback.

```
beginTx :: Pledge IO (RE Term) ()
beginTx = Pledge $ return
    ( ()
    , universe
    , Single (Atom "beginTx" (List []))
    , Or (finally (Atom "commit"   (List []))))
```

```
          (finally (Atom "rollback" (List []))) )

commit :: Pledge IO (RE Term) ()
commit = Pledge $ return
    ( ()
    , Or (Single (Atom "beginTx" (List [])))
         (Single (Atom "write"   Wildcard))     -- requires begin or write
    , Single (Atom "commit" (List []))
    , universe )

rollback :: Pledge IO (RE Term) ()
rollback = Pledge $ return
    ( (), universe
    , Single (Atom "rollback" (List []))
    , universe )

-- Good: begin, commit
committedTx = do { beginTx; commit }

-- Good: begin, rollback
rolledBackTx = do { beginTx; rollback }

-- Bad: transaction never closed
openTx = beginTx
```

`openTx` retains the full disjunction `finally(commit)` ∨ `finally(rollback)` as its residual future. `committedTx` and `rolledBackTx` fully discharge the obligation, yielding ¬∅. The disjunction is computed by the left-quotient: (`finally(commit)` ∨ `finally(rollback)`) \ `Single(commit)` reduces to ¬∅ because `commit` advances the `finally(commit)` branch to ¬∅.

9.3 Bounded Counter (GRE Instance)

A counter stored at heap address `addr` with maximum value `maxVal` must follow the protocol `init`·(`inc` | `dec`)*·`snapshot`. Increment is guarded by $h[\texttt{addr}] < \texttt{maxVal}$; decrement by $h[\texttt{addr}] > 0$. Neither constraint alone suffices: the RE alone would accept overflow sequences; the Presburger predicate alone would accept arbitrary event orders.

```
initCounter :: Addr → Pledge IO (GuardedRE Term) ()
initCounter addr = Pledge $ return
    ( ()
    , fromRE universe                                    -- pre: none
    , GuardedRE (PEq (ValAt addr) (Lit 0))
                (Single (Atom "init" (List [Num addr]))) -- post: heap[addr]=0
    , fromRE universe )                                  -- fut: none

increment :: Addr → Int → Pledge IO (GuardedRE Term) ()
increment addr maxVal = Pledge $ return
    ( ()
    , GuardedRE (PLt (ValAt addr) (Lit maxVal)) universe  -- pre: not at max
    , fromRE (Single (Atom "inc" (List [Num addr])))
```

```
    , fromPPred (PGe (ValAt addr) (Lit 0)) )              -- fut: stays >= 0

-- Good: init, two increments, one decrement, snapshot
normalUse = do
    initCounter 0; increment 0 10; increment 0 10; decrement 0; snapshot 0

-- Bad: 11 increments against max 10 -- PLt(h[0], 10) violated on 11th
overflow = do
    initCounter 0; replicateM_ 11 (increment 0 10); snapshot 0

-- Bad: decrement below zero -- PGt(h[0], 0) violated immediately
underflow = do { initCounter 0; decrement 0 }
```

Program	Residual *pre* (Presburger)	Residual *pre* (RE)
`normalUse`	**true**	$\neg\varnothing$
`overflow`	**false**	$\neg\varnothing$
`underflow`	**false**	$\neg\varnothing$

For `overflow`, composing eleven increments accumulates the conjunction $h[0] < 10 \wedge \neg(h[0] < 10)$, which `normalizePPred` collapses to `PFalse` without any solver call, pinpointing the arithmetic violation while the RE component remains $\neg\varnothing$.

9.4 Task Scheduling (WRE Tropical Instance)

Under the tropical (min-plus) semiring, weights represent minimum numbers of steps. Each `submit` carries a future condition requiring eventual `complete`.

```
type TSRE = WRE Tropical Term

submit :: Int → Pledge IO TSRE ()
submit taskId = Pledge $ return
    ( ()
    , WEps sone
    , WSingle (Tropical 1) (Atom "submit" (List [Num taskId]))
    , wFinally (Tropical 1) (Atom "complete" (List [Num taskId])) )

complete :: Int → Pledge IO TSRE ()
complete taskId = Pledge $ return
    ( ()
    , wPreviously (Tropical 1) (Atom "submit" (List [Num taskId]))  -- pre
    , WSingle (Tropical 1) (Atom "complete" (List [Num taskId]))
    , WEps sone )                                                   -- fut: done

-- Good: submit and complete two tasks
twoTasks = do { submit 1; complete 1; submit 2; complete 2 }

-- Good: choose cheaper of complete (cost 1) vs abort (cost 2) via WAdd
cheapestResolution = Pledge $ return
    ( (), WEps sone
    , WSingle (Tropical 1) (Atom "submit" (List [Num 1]))
```

```
    , WAdd (wFinally (Tropical 1) (Atom "complete" (List [Num 1])))
           (wFinally (Tropical 2) (Atom "abort"    (List [Num 1]))) )

-- Bad: task 2 never completed
missingComplete = do { submit 1; complete 1; submit 2 }
```

Program	Residual future	wNullable
twoTasks	WEps 0	0 steps
cheapestResolution	WAdd(...)	1 step
missingComplete	wFinally 1 (complete(2))	∞

For missingComplete, wNullable returns Tropical infinity, flagging that no finite trace discharges the obligation for task 2. For cheapestResolution, WAdd takes the minimum weight of the two alternatives: completing costs 1, aborting costs 2, so the minimum is 1.

9.5 Heap Memory (SL Instance)

Heap ownership is tracked via separation-logic predicates. Each alloc produces a Cell, and free consumes it.

```
alloc :: Addr → Val → Pledge IO SL ()
alloc addr val = Pledge $ return ((), Top, Cell addr val, Top)

free :: Addr → Val → Pledge IO SL ()
free addr val = Pledge $ return ((), Cell addr val, Emp, Top)

writeCell :: Addr → Val → Val → Pledge IO SL ()
writeCell addr old new = Pledge $ return ((), Cell addr old, Cell addr new, Top)

-- Good: alloc then immediately free
allocFree = do { alloc 0 42; free 0 42 }

-- Good: two disjoint cells allocated
allocTwo = do { alloc 0 1; alloc 1 2 }

-- Good: alloc, write, free -- full ownership cycle
fullCycle = do { alloc 0 0; writeCell 0 0 7; free 0 7 }

-- Bad: alloc two, free only one -- Cell 1 2 ownership remains in post
leakOne = do { alloc 0 1; alloc 1 2; free 0 1 }

-- Bad: read without ownership -- pre = Cell 0 42, never posted
readWithoutAlloc = Pledge $ return ((), Cell 0 42, Emp, Top)
```

Program	Residual *post*	Residual *pre*	Residual *fut*
allocFree	**emp**	⊤	⊤
allocTwo	Cell 0 1 ∗ Cell 1 2	⊤	⊤
fullCycle	**emp**	⊤	⊤
leakOne	Cell 1 2	⊤	⊤
readWithoutAlloc	**emp**	Cell 0 42	⊤

For `allocTwo`, the combined *post* is `Cell 0 1 ∗ Cell 1 2`: two disjointly owned cells composed by separating conjunction, correctly reflecting that the two cells reside at distinct addresses. For `leakOne`, the *post* retains `Cell 1 2`, naming the unreleased ownership at address 1 precisely. Unlike the RE examples where a non-$\top$ *fut* is a definite violation, in the SL instance the caller inspects *post* to decide whether retained ownership constitutes a leak; *fut* = $\top$ means no future obligation was stated, but outstanding *post* ownership is still visible.

10 Monad Laws

Every monad must satisfy three equational laws. **`Pledge`**'s four annotation fields do not all satisfy the laws under ordinary equality: *pre* follows the Hoare-rule discipline of Equation (1), which deliberately departs from left-identity when a precondition violation is detected (the violation must remain visible after further binding). What we prove, and mechanise in Lean 4, is that the triple (*ret*, *post*, *fut*) satisfies the three laws exactly. The *pre* field is carried alongside as a separate contract-checking annotation, analogous to a writer-monad log that is inspected but not required to satisfy the monad laws on its own.

The laws hold assuming the twelve axioms in Table 5, which we state abstractly over the **`Composable`** algebra.

Table 5. The twelve **`Composable`** axioms required to prove the monad laws for the (*ret*, *post*, *fut*) triple.

Label	Axiom	Used for
S1	$\varepsilon \cdot a = a$	Law 1, post
S2	$a \cdot \varepsilon = a$	Law 2, post
S3	$(a \cdot b) \cdot c = a \cdot (b \cdot c)$	Law 3, post
C1	$a \sqcap b = b \sqcap a$	Law 2, pre / fut
C2	$(a \sqcap b) \sqcap c = a \sqcap (b \sqcap c)$	Law 3, pre / fut
C3	$\top \sqcap a = a$	Laws 1, 2
L1	$r \setminus \varepsilon = r$	Law 2, fut
L2	$\top \setminus r = \top$	Law 1, fut
L3	$r \setminus (a \cdot b) = (r \setminus a) \setminus b$	Law 3, fut
L4	$(a \sqcap b) \setminus c = (a \setminus c) \sqcap (b \setminus c)$	Law 3, fut
R1	$a \,/\, \varepsilon = a$	Law 1, pre
R2	$\top \,/\, r = \top$	Law 2, pre
R3	$(a \,/\, b) \,/\, c = a \,/\, (c \cdot b)$	Law 3, pre
R4	$(a \sqcap b) \,/\, c = (a \,/\, c) \sqcap (b \,/\, c)$	Law 3, pre

We verify all three laws using these axioms. In each case we show that both sides of the law produce equal (*ret*, *post*, *fut*) triples.

Left identity: `pure x >>= f = f x`. Let p = `pure x` = $(x, \top, \varepsilon, \top)$ and $q = f\,x$. By Equations (1)–(3):

$$\begin{aligned} \mathit{ret} &: \mathit{ret}(q).\checkmark \\ \mathit{post} &: \varepsilon \cdot \mathit{post}(q) = \mathit{post}(q). \quad \text{(S1)}\checkmark \\ \mathit{fut} &: (\top \setminus \mathit{post}(q)) \sqcap \mathit{fut}(q) = \top \sqcap \mathit{fut}(q) = \mathit{fut}(q). \quad \text{(L2, C3)}\checkmark \end{aligned}$$

For the future, $\mathit{pre}(p) = \top$ is the right-quotient's absorbing element: $\mathit{pre}(q) \,/\, \varepsilon = \mathit{pre}(q)$ (R1), so $\mathit{pre} = \top \sqcap \mathit{pre}(q) = \mathit{pre}(q)$ (C3). ✓

Right identity: `e >>= pure = e`. Let $q = \mathsf{pure}(ret(e)) = (ret(e), \top, \varepsilon, \top)$.

$$ret : ret(e).\checkmark$$
$$post : post(e) \cdot \varepsilon = post(e). \quad \text{(S2)}\checkmark$$
$$fut : (fut(e) \setminus \varepsilon) \sqcap \top = fut(e) \sqcap \top = \top \sqcap fut(e) = fut(e). \quad \text{(L1, C1, C3)}\checkmark$$

Associativity. Let $r_1 = ret(e)$, $r_2 = ret(f\, r_1)$. Both sides yield $ret(g\, r_2)$ and $post(e) \cdot post(f\, r_1) \cdot post(g\, r_2)$ (by S3). For the future, unfolding Equation (3) on the LHS and applying L4 then L3 gives:

$$\big((fut(e) \setminus post(f\, r_1)) \setminus post(g\, r_2)\big) \sqcap (fut(f\, r_1) \setminus post(g\, r_2)) \sqcap fut(g\, r_2)$$

which equals the RHS by C2 and L3. ✓ For the precondition, applying R4 then R3 gives the same result on both sides by C2. ✓

Lean 4 mechanisation. The three law proofs are mechanised in Lean 4 in the file `Formalization/PledgeMonad` using an abstract **`Composable`** structure parametrised over the twelve axioms of Table 5. The Lean file carries no `sorry`; all proofs are term-mode derivations from the stated axioms. A companion file `Formalization/REComposableLaws.lean` verifies that the RE instance satisfies all twelve axioms.

11 Related Work

Future conditions and temporal verification. The proposal this paper implements is due to Song et al. [2025], who introduce future conditions as a first-class extension to pre/post contracts at the level of a formal system. Song et al. [2024] apply a related RE encoding of temporal properties in ProveNFix to guide automated program repair. Pledge's contribution is a concrete, runnable library realisation: a monad transformer, a derivative-based discharge mechanism, and four **`Composable`** instances that neither prior paper instantiates.

Pre/post-condition contracts in Haskell. Liquid Haskell [Vazou et al. 2014] expresses pre/post-conditions as refinement types verified by an SMT solver. Findler and Felleisen [Findler and Felleisen 2002] establish runtime contract discipline for higher-order functions. Swierstra and Baanen [Swierstra and Baanen 2019] embed Hoare triples as an indexed monad, propagating pre/post annotations through monadic bind analogously to **`Pledge`**. Pledge complements these by adding a temporal future condition without requiring type-system extensions.

Parameterised and graded monads. Atkey [2009] introduces parameterised monads for Hoare-style state tracking; Katsumata [2014] generalises this to graded monads; Orchard and Yoshida [2016] shows effect systems and session types are instances. Pledge carries obligations as value-level annotation fields rather than indexing by effect type at the type level, preserving compatibility with the standard `Monad` class and **`do`**-notation.

Resource management in Haskell. The `bracket` combinator lexically scopes cleanup. `ResourceT` [Snoyman 2011], `Acquire`, and `Managed` thread finaliser queues through every bind. All three handle the cleanup half of a resource obligation but cannot express protocol or quantitative obligations, which require explicit future-condition propagation across bind steps.

Linear types. Linear Haskell [Bernardy et al. 2018] enforces "consume exactly once". Linear types enforce structural obligations (use at most once), whereas future conditions enforce temporal obligations (use in a specified order, or with a specified weight). A hybrid combining both could achieve stronger guarantees than either alone.

Session types. Session types [Gay and Hole 2005; Honda 1993] enforce communication protocols at the type level. The `sessiontypes` library [van Walree 2017] realises this via an indexed monad.

Pledge is more lightweight: it operates within the standard `Monad` class, admits quantitative and heap-ownership obligations that session types do not, and is applicable beyond message-passing.

Runtime monitoring. Runtime verification [Leucker and Schallhart 2009] checks temporal properties by instrumenting a concrete execution. Pledge propagates obligations in the specification layer before any effectful run, without requiring a concrete execution, at the cost of not itself catching divergence between the specification and the real code it shadows.

Brzozowski derivatives. Derivatives of regular expressions [Brzozowski 1964] have found application in parsing [Might et al. 2011; Owens et al. 2009] and model checking. Equation (4) extends this to complement; our left/right-quotient operators are a novel application to future-condition propagation in a monadic setting.

12 Discussion and Future Work

Decidability. Equality of normalised regular expressions is decidable, so the left-quotient computation terminates whenever the derivative sequence is finite, which holds for the regular languages used in our examples. Extending to context-free or ω-regular specifications remains future work.

Closing the gap between specification and real effects. The monad transformer structure allows live runtime values to be embedded in annotations (as in the file-handle example of §3.4), but **`Pledge`** itself does not check that the shadow specification stays consistent with the real code as it evolves. Lifting **`Pledge`** into an effect-handler framework (`effectful` or `polysemy`), so that a **`Pledge`** annotation is attached directly to each operation's real effect handler and updated automatically as the effect executes, would close this gap.

Toward solver-backed SL discharge. §8 is explicit that the SL instance's **`leftQuotient`** records the magic wand symbolically. Wiring residual SL predicates into a decision procedure (Z3 via `sbv` or a dedicated SL prover such as Smallfoot) so that **`Wand`** nodes are checked for satisfiability, rather than merely recorded, is the concrete next step toward a soundness statement for SL analogous to Theorem 5.2.

Parallel composition. The **`Pledge`** monad is inherently sequential; concurrent programs require a parallel combinator $e_1 \parallel e_2$. In the RE instance this corresponds to the shuffle product $r_1 \sqcup\!\sqcup r_2$; the SL instance admits a parallel connective via the concurrent separation logic frame rule [O'Hearn 2007]. Extending **`Composable`** with a `parallel` operation is a natural next step.

Closed-world alphabet assumption. The `firstWith` function uses $\Sigma_0 = atoms(r_1) \cup atoms(r_2)$ as its working alphabet. This is sound under the closed-world assumption that every relevant event is mentioned in the two operands. A fully open-world implementation would accept an explicit Σ declaration.

13 Conclusion

We have presented *Pledge*, a Haskell library that realises Song et al. [2025]'s proposal of future conditions as a concrete monad transformer: a **`Pledge`** `m eff a` computation in underlying monad `m` that carries three annotation fields $(pre, post, fut) :: eff$, propagated algebraically through monadic bind via left- and right-quotient operations.

The central insight is that trace-based, arithmetic, quantitative, and heap-ownership obligations are structurally identical at the level of a six-operation **`Composable`** algebra. All four instances (RE, GRE, WRE, SL) share a single bind formula, and the diagnostic signal is uniform: a non-trivial residual in *pre* or *fut* identifies exactly which obligation remains unmet and why, a diagnostic we

prove sound for the RE instance (Theorem 5.2). The monad laws for the (*ret*, *post*, *fut*) triple are verified mechanically in Lean 4 from twelve algebraic axioms.

Pledge requires no language extensions beyond standard type classes, composes via a standard `Monad` instance, and integrates with real effectful code through the monad transformer design: when `m = IO`, runtime values such as file handles or thread identifiers can be captured inside the underlying action and embedded directly into the annotation fields. Tightening the connection between specification and runtime execution — through effect-handler integration and solver-backed SL discharge — is the paper's main direction for future work.

References

Robert Atkey. 2009. Parameterised Notions of Computation. *Journal of Functional Programming* 19, 3–4 (2009), 335–376. doi:10.1017/S095679680900728X

Jean-Philippe Bernardy, Mathieu Boespflug, Ryan R. Newton, Simon Peyton Jones, and Arnaud Spiwack. 2018. Linear Haskell: Practical Linearity in a Higher-Order Polymorphic Language. *Proceedings of the ACM on Programming Languages* 2, POPL, Article 5 (2018), 29 pages. doi:10.1145/3158093

Janusz A. Brzozowski. 1964. Derivatives of Regular Expressions. *J. ACM* 11, 4 (1964), 481–494. doi:10.1145/321239.321249

Robert Bruce Findler and Matthias Felleisen. 2002. Contracts for Higher-Order Functions. In *Proceedings of the 7th ACM SIGPLAN International Conference on Functional Programming (ICFP)*. 48–59. doi:10.1145/581478.581484

Simon J. Gay and Malcolm Hole. 2005. Subtyping for Session Types in the Pi Calculus. *Acta Informatica* 42, 2-3 (2005), 191–225. doi:10.1007/s00236-005-0177-z

Kohei Honda. 1993. Types for Dyadic Interaction. In *Proceedings of the 4th International Conference on Concurrency Theory (CONCUR) (Lecture Notes in Computer Science, Vol. 715)*. 509–523. doi:10.1007/3-540-57208-2_35

Shin-ya Katsumata. 2014. Parametric Effect Monads and Semantics of Effect Systems. *ACM SIGPLAN Notices* 49, 1 (2014), 633–646. doi:10.1145/2578855.2535846

Martin Leucker and Christian Schallhart. 2009. A Brief Account of Runtime Verification. *Journal of Logic and Algebraic Programming* 78, 5 (2009), 293–303. doi:10.1016/j.jlap.2008.08.004

Matthew Might, David Darais, and Daniel Spiewak. 2011. Parsing with Derivatives: A Functional Pearl. In *Proceedings of the 16th ACM SIGPLAN International Conference on Functional Programming (ICFP)*. 189–195. doi:10.1145/2034773.2034801

Peter W. O'Hearn. 2007. Resources, Concurrency, and Local Reasoning. *Theoretical Computer Science* 375, 1–3 (2007), 271–307. doi:10.1016/j.tcs.2006.12.035

Dominic Orchard and Nobuko Yoshida. 2016. Effects as Sessions, Sessions as Effects. In *Proceedings of the 43rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 568–581. doi:10.1145/2837614.2837634

Scott Owens, John Reppy, and Aaron Turon. 2009. Regular-expression Derivatives Re-examined. *Journal of Functional Programming* 19, 2 (2009), 173–190. doi:10.1017/S0956796808007090

Michael Snoyman. 2011. conduit: Streaming Data Processing. https://hackage.haskell.org/package/conduit. Haskell package on Hackage; introduces the `ResourceT` transformer.

Yahui Song, Darius Foo, and Wei-Ngan Chin. 2025. Specifying and Verifying Future Conditions. In *Static Analysis — 32nd International Symposium, SAS 2025 (Lecture Notes in Computer Science, Vol. 16100)*. Springer, 90–112. doi:10.1007/978-3-032-07106-4_5

Yahui Song, Xiang Gao, Wenhua Li, Wei-Ngan Chin, and Abhik Roychoudhury. 2024. ProveNFix: Temporal Property-Guided Program Repair. *Proceedings of the ACM on Software Engineering* 1, FSE, Article 11 (2024), 23 pages. doi:10.1145/3643737

Wouter Swierstra and Tim Baanen. 2019. A Predicate Transformer Semantics for Effects (Functional Pearl). *Proceedings of the ACM on Programming Languages* 3, ICFP, Article 103 (2019), 26 pages. doi:10.1145/3341707

Ferdinand van Walree. 2017. sessiontypes: Session Types for Haskell. https://hackage.haskell.org/package/sessiontypes.

Niki Vazou, Eric L. Seidel, Ranjit Jhala, Dimitrios Vytiniotis, and Simon Peyton Jones. 2014. Refinement Types for Haskell. In *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming (ICFP)*. 269–282. doi:10.1145/2628136.2628161